

PRAKTIKUM SCIKIT LEARN (Laporan Praktikum Kontrol Cerdas)

Nama : Muchamad Yusuf Revaldo

Nim : 234308078

Kelas : TKA-6C

Akun Github : <https://github.com/yusufrevaldo>

Abstrak

Praktikum ini membahas implementasi machine learning menggunakan pustaka Scikit-learn pada bahasa pemrograman Python dalam membangun sistem klasifikasi berbasis data. Tujuan praktikum adalah memahami konsep dasar klasifikasi, tahapan preprocessing data, proses training dan testing model, serta penyimpanan dataset dan model menggunakan file berekstensi *.pickle*. Tahapan yang dilakukan meliputi pembacaan dataset, pemisahan fitur dan label, pembagian data latih dan data uji menggunakan metode *train_test_split*, pelatihan model dengan fungsi *fit()*, serta proses prediksi menggunakan fungsi *predict()*. Hasil percobaan menunjukkan bahwa model klasifikasi mampu mengenali pola data berdasarkan fitur numerik yang diberikan, termasuk pada kasus klasifikasi posisi pengguna (berdiri atau duduk). Penyimpanan dataset dan model dalam format *.pickle* terbukti meningkatkan efisiensi karena memungkinkan penggunaan kembali model tanpa perlu melakukan pelatihan ulang. Secara keseluruhan, praktikum ini memberikan pemahaman sistematis mengenai alur kerja machine learning mulai dari pengolahan data hingga implementasi model prediktif.

Kata kunci: *Machine Learning, Klasifikasi, Scikit-Learn, Python, Preprocessing Data, Model Pickle, Train Test Split.*

1. Pendahuluan

Machine learning merupakan salah satu pendekatan komputasi yang berkembang pesat dalam berbagai bidang ilmu karena kemampuannya dalam mengolah data dan menemukan pola tersembunyi secara otomatis. Pendekatan ini memungkinkan sistem untuk belajar dari data tanpa harus diprogram secara eksplisit untuk setiap kemungkinan kasus. Dalam implementasinya menggunakan bahasa pemrograman Python, machine learning banyak diterapkan dalam analisis data, klasifikasi, regresi, dan pengenalan pola berbasis statistik (Abraham dkk., 2014).

Scikit-Learn merupakan pustaka machine learning berbasis Python yang menyediakan berbagai algoritma pembelajaran statistik seperti klasifikasi, regresi, clustering, serta reduksi dimensi. Pustaka ini dirancang dengan antarmuka yang sederhana dan konsisten sehingga memudahkan pengguna dalam membangun model prediktif secara efisien. Selain itu, Scikit-

Learn terintegrasi dengan pustaka ilmiah Python lainnya seperti NumPy dan SciPy sehingga mendukung proses komputasi numerik yang optimal (Abraham dkk., 2014).

Penerapan Scikit-Learn dalam berbagai penelitian menunjukkan bahwa pustaka ini dapat digunakan dalam berbagai bidang, salah satunya dalam analisis teks dan deteksi berita palsu melalui teknik data mining dan klasifikasi seperti Naive Bayes dan Support Vector Machine. Penggunaan vectorizer seperti TF-IDF dalam Scikit-Learn memungkinkan data teks diubah menjadi representasi numerik sehingga dapat diproses oleh algoritma klasifikasi (Nursakinah, t.t.).

Berdasarkan uraian tersebut, praktikum Scikit-Learn bertujuan untuk memberikan pemahaman mengenai konsep dasar machine learning serta implementasi algoritma menggunakan Python. Melalui kegiatan ini, mahasiswa diharapkan mampu melakukan proses prapemrosesan data, pelatihan model, evaluasi performa, serta interpretasi hasil analisis secara sistematis sesuai dengan kaidah data mining dan machine learning (Nursakinah, t.t.).

2. Tujuan

- 1 Memahami konsep dasar machine learning khususnya pada metode klasifikasi.
- 2 Mempelajari penggunaan library scikit-learn dalam proses training model.
- 3 Melakukan preprocessing dan normalisasi data sebelum proses klasifikasi.
- 4 Membangun model klasifikasi gesture tangan menggunakan algoritma pada scikit-learn.

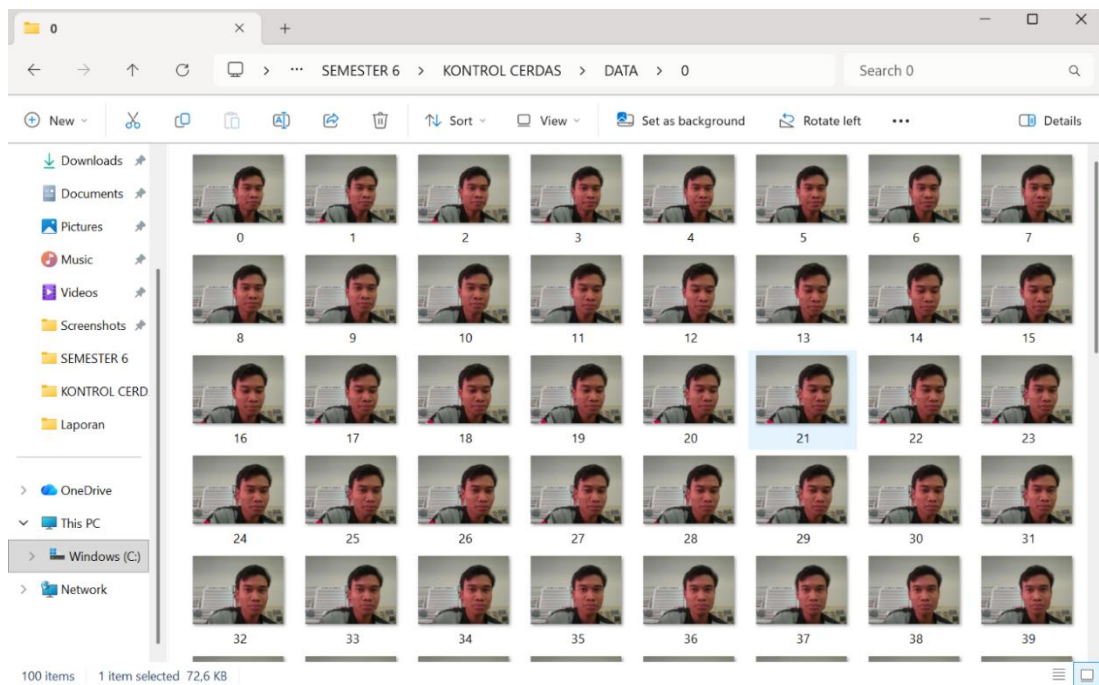
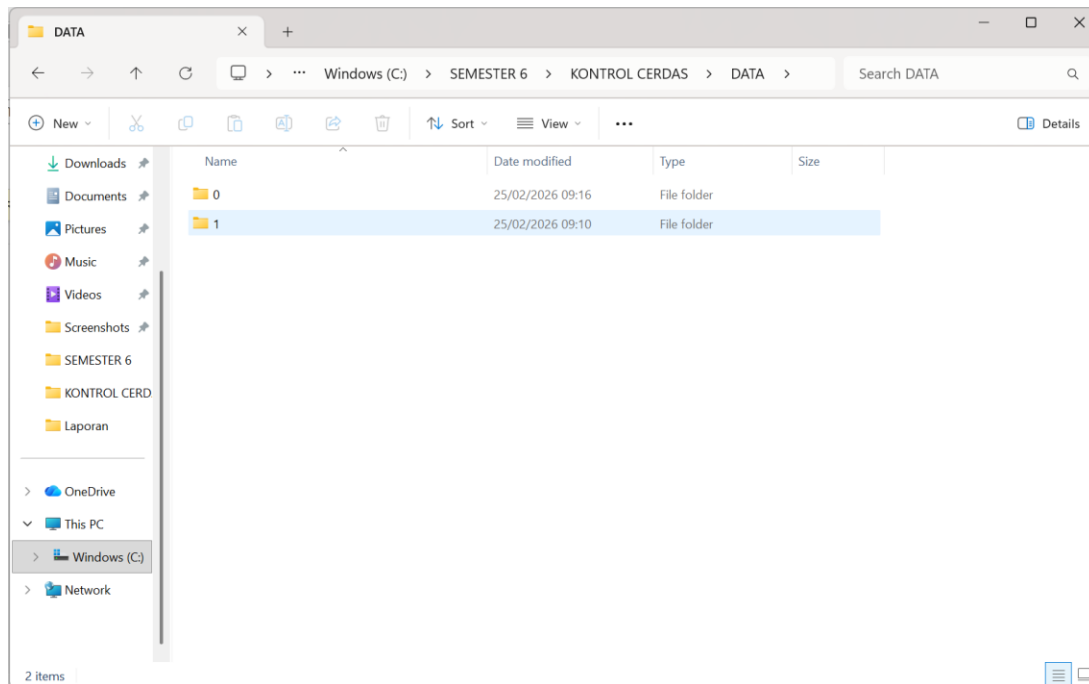
3. Manfaat

- 1 Mahasiswa memahami tahapan kerja machine learning secara sistematis.
- 2 Mahasiswa mampu mengolah dataset dan melakukan ekstraksi fitur sebelum training.
- 3 Mahasiswa dapat membangun dan menyimpan model klasifikasi menggunakan scikit-learn.

4. Hasil Percobaan

A. Percobaan Pendaahuluan A

Program Dan Hasil Running Program



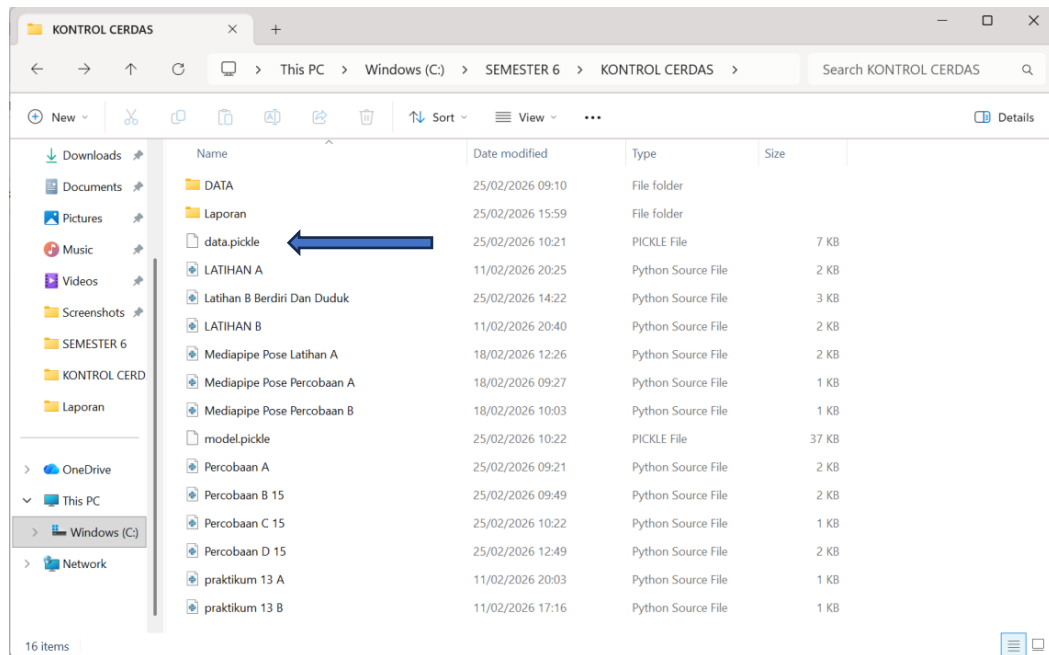
```

1  import os
2  import cv2
3
4  DATA_DIR = r'C:\SEMESTER 6\KONTROL CERDAS\DATA'
5
6  if not os.path.exists(DATA_DIR):
7      os.makedirs(DATA_DIR)
8
9  number_of_classes = 2
10 dataset_size = 100
11
12 cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
13
14 if not cap.isOpened():
15     print("Kamera tidak terdeteksi!")
16     exit()
17
18 for j in range(number_of_classes):
19
20     class_path = os.path.join(DATA_DIR, str(j))
21     os.makedirs(class_path, exist_ok=True)
22
23     print('Collecting data for class {}'.format(j))
24     print('Tekan Q untuk mulai capture')
25
26 while True:
27     ret, frame = cap.read()
28     if not ret:
29         print("Gagal baca kamera")
30         break
31
32     cv2.putText(frame, text: 'Ready? Press Q!', org: (100, 50), cv2.FONT_HERSHEY_SIMPLEX, fontScale: 1, color: (0, 255, 0), thickness: 3)
33
34     cv2.imshow( winname: 'frame', frame)
35
36     if cv2.waitKey(1) & 0xFF == ord('q'):
37         break
38
39 counter = 0
40 while counter < dataset_size:
41     ret, frame = cap.read()
42     if not ret:
43         print("Gagal baca kamera")
44         break
45
46     cv2.imshow( winname: 'frame', frame)
47     cv2.imwrite(os.path.join(class_path, f'{counter}.jpg'), frame)
48     cv2.waitKey(50)
49     counter += 1
50
51 print("Selesai ambil data!")
52
53 cap.release()
54 cv2.destroyAllWindows()

```

B. Percobaan B Data.Pickle

Program Dan Hasil Running Program



```
1 import os
2 import pickle
3 import mediapipe as mp
4 import cv2
5
6 mp_hands = mp.solutions.hands
7 hands = mp_hands.Hands(static_image_mode=True, max_num_hands=1)
8
9 DATA_DIR = r'C:\SEMESTER 6\KONTROL CERDAS\DATA'
10
11 data = []
12 labels = []
13
14 for dir_ in os.listdir(DATA_DIR):
15     for img_path in os.listdir(os.path.join(DATA_DIR, dir_)):
16
17         data_aux = []
18         x_ = []
19         y_ = []
20
21         img = cv2.imread(os.path.join(DATA_DIR, dir_, img_path))
22
23         if img is None:
24             continue
25
```

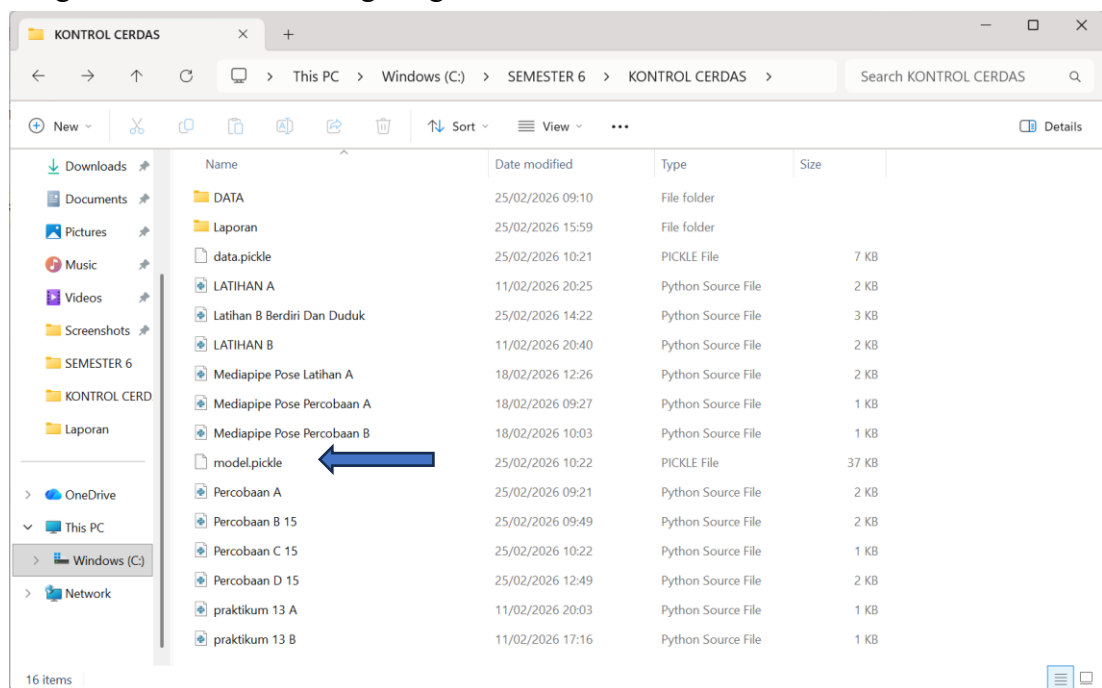
```

25
26     img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
27     results = hands.process(img_rgb)
28
29     if results.multi_hand_landmarks:
30         for hand_landmarks in results.multi_hand_landmarks:
31
32             for landmark in hand_landmarks.landmark:
33                 x_.append(landmark.x)
34                 y_.append(landmark.y)
35
36             for landmark in hand_landmarks.landmark:
37                 data_aux.append(landmark.x - min(x_))
38                 data_aux.append(landmark.y - min(y_))
39
40         data.append(data_aux)
41         labels.append(dir_)
42
43     with open(r'C:\SEMESTER 6\KONTROL CERDAS\data.pickle', 'wb') as f:
44         pickle.dump({'data': data, 'labels': labels}, f)
45
46     print("Data berhasil disimpan!")

```

C. Percobaan C Model.Pickle

Program Dan Hasil Running Program



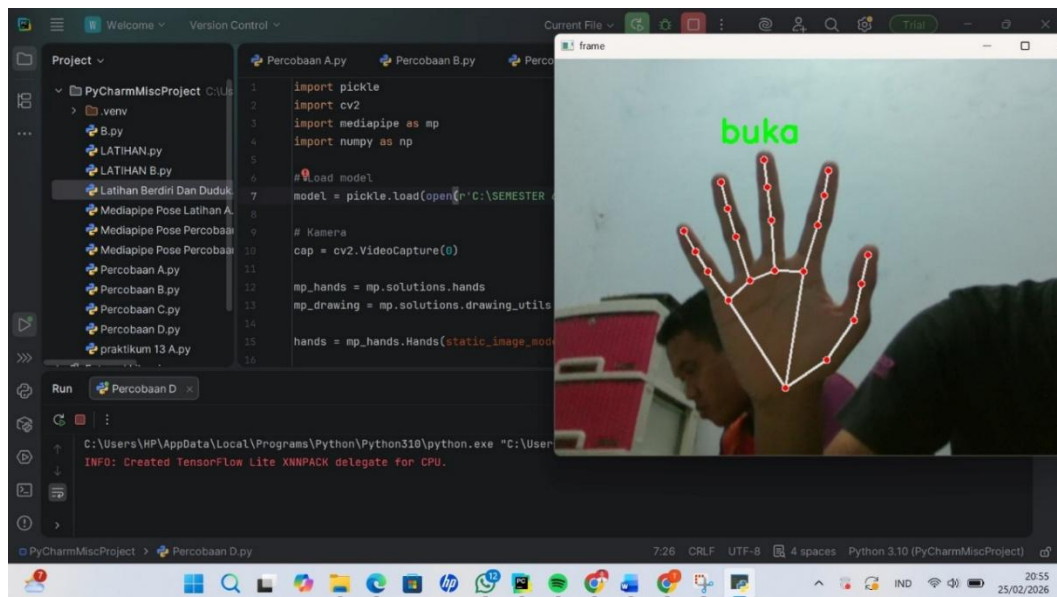
```

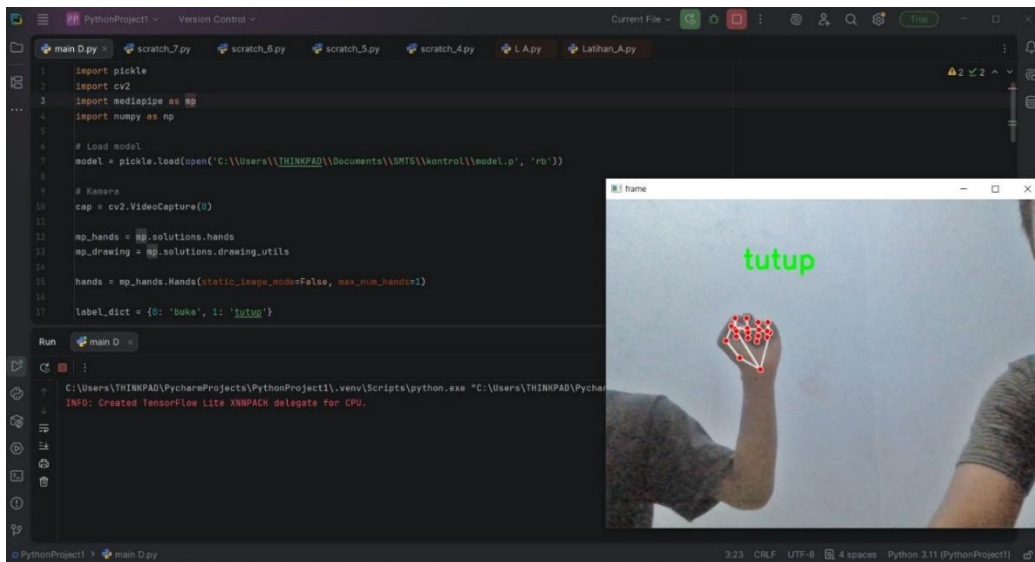
1 import pickle
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
5 import numpy as np
6
7 data_dict = pickle.load(open(r'C:\SEMESTER 6\KONTROL CERDAS\data.pickle', 'rb'))
8
9 data = np.array(data_dict['data'])
10 labels = np.array(data_dict['labels'])
11
12 x_train, x_test, y_train, y_test = train_test_split(*arrays: data, labels, test_size=0.1, shuffle=True, stratify=labels)
13
14 model = RandomForestClassifier()
15 model.fit(x_train, y_train)
16
17 y_predict = model.predict(x_test)
18 score = accuracy_score(y_predict, y_test)
19
20 print('{}% of samples were classified correctly!'.format(score * 100))
21
22 with open(r'C:\SEMESTER 6\KONTROL CERDAS\model.pickle', 'wb') as f:
23     pickle.dump(model, f)
24
25 print("Model berhasil disimpan!")

```

D. Percobaan D Model.Pickle

Program Dan Hasil Running Program





```

1 import pickle
2 import cv2
3 import mediapipe as mp
4 import numpy as np
5
6 # Load model
7 model = pickle.load(open('C:\\SEMESTER 6\\KONTROL CERDAS\\model.pickle', 'rb'))
8
9 # Kamera
10 cap = cv2.VideoCapture(0)
11
12 mp_hands = mp.solutions.hands
13 mp_drawing = mp.solutions.drawing_utils
14
15 hands = mp_hands.Hands(static_image_mode=False, max_num_hands=1)
16
17 label_dict = {0: 'buka', 1: 'tutup'}
18
19 while True:
20     data_aux = []
21     x_ = []
22     y_ = []
23
24     ret, frame = cap.read()
25     if not ret:
26         break
27
28     frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
29     results = hands.process(frame_rgb)
30
31     if results.multi_hand_landmarks:
32         for hand_landmarks in results.multi_hand_landmarks:
33             mp_drawing.draw_landmarks(
34                 frame,
35                 hand_landmarks,
36                 mp_hands.HAND_CONNECTIONS
37             )
38
39             for lm in hand_landmarks.landmark:
40                 x_.append(lm.x)
41                 y_.append(lm.y)
42
43             for lm in hand_landmarks.landmark:
44                 data_aux.append(lm.x - min(x_))
45                 data_aux.append(lm.y - min(y_))
46
47             prediction = model.predict([np.asarray(data_aux)])
48             predicted_character = label_dict[int(prediction[0])]
49
50             cv2.putText(frame, predicted_character, (100, 150), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
51
52     cv2.imshow('frame', frame)
53     if cv2.waitKey(1) & 0xFF == ord('q'):
54         break
55
56 cap.release()
57 cv2.destroyAllWindows()

```

```

26         break
27
28     frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
29     results = hands.process(frame_rgb)
30
31     if results.multi_hand_landmarks:
32         for hand_landmarks in results.multi_hand_landmarks:
33             mp_drawing.draw_landmarks(
34                 frame,
35                 hand_landmarks,
36                 mp_hands.HAND_CONNECTIONS
37             )
38
39             for lm in hand_landmarks.landmark:
40                 x_.append(lm.x)
41                 y_.append(lm.y)
42
43             for lm in hand_landmarks.landmark:
44                 data_aux.append(lm.x - min(x_))
45                 data_aux.append(lm.y - min(y_))
46
47             prediction = model.predict([np.asarray(data_aux)])
48             predicted_character = label_dict[int(prediction[0])]
49
50             cv2.putText(frame, predicted_character, (100, 150), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
51
52     cv2.imshow('frame', frame)
53     if cv2.waitKey(1) & 0xFF == ord('q'):
54         break
55
56 cap.release()
57 cv2.destroyAllWindows()

```



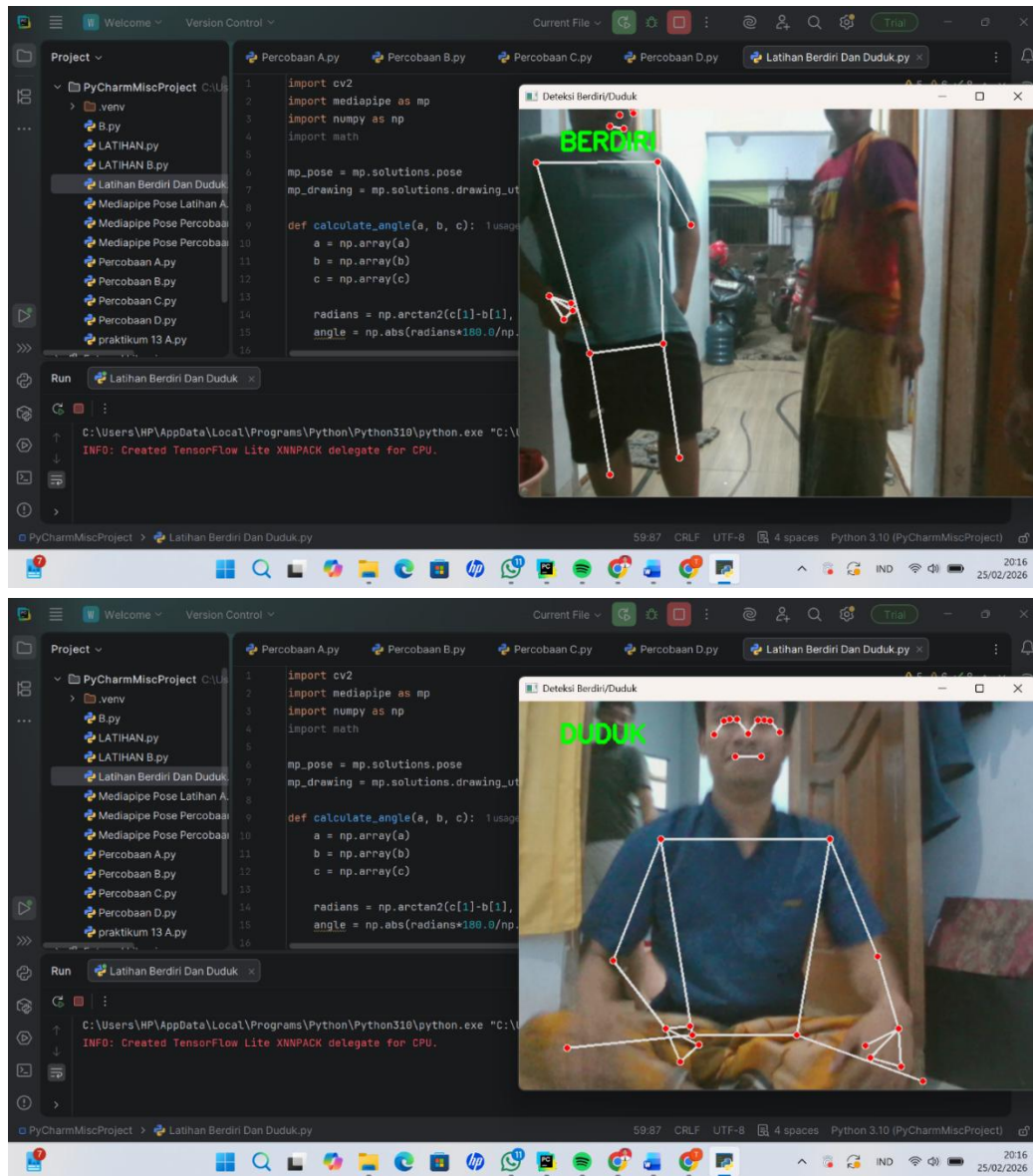
```

50
51         cv2.putText(frame, predicted_character,
52                       org: (200, 100),
53                       cv2.FONT_HERSHEY_SIMPLEX,
54                       fontScale: 1.3, color: (0, 255, 0), thickness: 3)
55
56     cv2.imshow( winname: 'frame', frame)
57
58     if cv2.waitKey(1) & 0xFF == ord('q'):
59         break
60
61 cap.release()
62 cv2.destroyAllWindows()

```

E. Percobaan Latihan user berdiri atau duduk

Program Dan Hasil Running Program



```

1  import cv2
2  import mediapipe as mp
3  import numpy as np
4  import math
5
6  mp_pose = mp.solutions.pose
7  mp_drawing = mp.solutions.drawing_utils
8
9  def calculate_angle(a, b, c): 1 usage
10     a = np.array(a)
11     b = np.array(b)
12     c = np.array(c)
13
14     radians = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1], a[0]-b[0])
15     angle = np.abs(radians*180.0/np.pi)
16
17     if angle > 180.0:
18         angle = 360 - angle
19
20     return angle
21
22 cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
23
24 with mp_pose.Pose(min_detection_confidence=0.5,min_tracking_confidence=0.5) as pose:
25
26     while cap.isOpened():
27         ret, frame = cap.read()
28         if not ret:
29             break
30
31         image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
32         image.flags.writeable = False
33         results = pose.process(image)
34
35         image.flags.writeable = True
36         image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
37
38         try:
39             landmarks = results.pose_landmarks.landmark
40
41             hip = [landmarks[mp_pose.PoseLandmark.RIGHT_HIP.value].x,
42                   landmarks[mp_pose.PoseLandmark.RIGHT_HIP.value].y]
43
44             knee = [landmarks[mp_pose.PoseLandmark.RIGHT_KNEE.value].x,
45                   landmarks[mp_pose.PoseLandmark.RIGHT_KNEE.value].y]
46
47             ankle = [landmarks[mp_pose.PoseLandmark.RIGHT_ANKLE.value].x,
48                    landmarks[mp_pose.PoseLandmark.RIGHT_ANKLE.value].y]
49
50             angle = calculate_angle(hip, knee, ankle)
51
52             if angle > 160:
53                 status = "BERDIRI"
54             elif angle < 140:
55                 status = "DUDUK"
56             else:
57                 status = "TRANSISI"
58
59             cv2.putText(image, status, org=(50, 50), cv2.FONT_HERSHEY_SIMPLEX, fontScale=1, color=(0, 255, 0), thickness=3)
60         except:
61             pass
62
63         mp_drawing.draw_landmarks(
64             image,
65             results.pose_landmarks,
66             mp_pose.POSE_CONNECTIONS)
67
68         cv2.imshow(winname: 'Deteksi Berdiri/Duduk', image)
69         if cv2.waitKey(10) & 0xFF == ord('q'):
70             break
71     cap.release()
72     cv2.destroyAllWindows()

```

5. Analisis Program

Berdasarkan hasil percobaan yang telah dilakukan, program pada praktikum Scikit-Learn ini secara umum bertujuan untuk membangun sistem klasifikasi berbasis machine learning menggunakan data hasil ekstraksi fitur, kemudian menyimpannya dalam bentuk file *.pickle* agar dapat digunakan kembali tanpa perlu melakukan proses training ulang.

Pada percobaan pendahuluan (Percobaan A), program berfungsi untuk membaca dataset serta mempersiapkan data sebelum dilakukan proses pelatihan model. Tahapan yang dilakukan meliputi pemanggilan library seperti NumPy dan Scikit-Learn, pembacaan file dataset, serta pemisahan antara data fitur (X) dan label (y). Pada tahap ini juga dilakukan pembagian data menjadi data latih dan data uji menggunakan fungsi `train_test_split`. Proses ini penting agar model dapat diuji performanya terhadap data yang belum pernah dilihat sebelumnya sehingga dapat mengukur tingkat generalisasi model.

Pada Percobaan B (*data.pickle*), program difokuskan pada proses penyimpanan dataset yang telah diproses ke dalam file *pickle*. File *pickle* digunakan untuk menyimpan objek Python dalam bentuk biner sehingga dapat dimuat kembali tanpa perlu melakukan ekstraksi atau preprocessing ulang. Tahap ini menunjukkan efisiensi dalam pengelolaan data, terutama jika dataset berukuran besar atau proses preprocessing cukup kompleks.

Pada Percobaan C (*model.pickle*), dilakukan proses training model klasifikasi menggunakan algoritma dari Scikit-Learn. Data yang telah dipisahkan sebelumnya digunakan untuk melatih model menggunakan metode `fit()`. Setelah model selesai dilatih, model tersebut disimpan dalam file *model.pickle*. Penyimpanan model ini bertujuan agar model yang sudah memiliki parameter terlatih dapat digunakan kembali untuk proses prediksi tanpa perlu mengulangi proses pelatihan dari awal.

Pada Percobaan D, file *model.pickle* yang telah dibuat sebelumnya dimuat kembali menggunakan fungsi `pickle.load()`. Program kemudian melakukan proses prediksi menggunakan metode `predict()` terhadap data baru yang diberikan. Tahapan ini menunjukkan implementasi nyata dari sistem machine learning, di mana model yang telah dilatih dapat digunakan untuk mengenali atau mengklasifikasikan input baru secara otomatis.

Pada percobaan latihan klasifikasi posisi user (berdiri atau duduk), sistem memanfaatkan fitur hasil ekstraksi pose atau gesture sebagai input model. Data posisi tubuh diubah menjadi representasi numerik sebelum diproses oleh algoritma klasifikasi. Model kemudian menentukan kelas berdasarkan pola yang telah dipelajari sebelumnya. Hal ini

menunjukkan bahwa sistem machine learning bekerja dengan cara mengenali pola dari data latih, bukan berdasarkan aturan yang ditentukan secara manual.

6. Kesimpulan

- 1 Scikit-Learn memudahkan proses pembangunan model machine learning karena memiliki struktur fungsi yang sistematis seperti `fit()`, `predict()`, dan `train_test_split()`.
- 2 Proses preprocessing dan pembagian dataset sangat penting untuk menghasilkan model yang memiliki kemampuan generalisasi yang baik.
- 3 Penyimpanan dataset dan model dalam bentuk file *.pickle* meningkatkan efisiensi karena model dapat digunakan kembali tanpa perlu melakukan training ulang.
- 4 Model klasifikasi yang dibangun mampu mengenali pola dari data gesture atau posisi tubuh berdasarkan fitur numerik yang diberikan.

7. Saran

Pada praktikum selanjutnya sebaiknya ditambahkan perhitungan akurasi dan confusion matrix agar hasil klasifikasi bisa diketahui tingkat keberhasilannya secara jelas. Dataset yang digunakan juga perlu diperbanyak supaya model bisa mengenali pola dengan lebih baik dan tidak mudah salah saat menerima data baru. Selain itu, bisa dicoba menggunakan beberapa algoritma lain untuk dibandingkan hasilnya sehingga dapat diketahui metode mana yang paling cocok. Jika memungkinkan, sistem juga bisa dikembangkan ke tahap real-time agar lebih menarik dan aplikatif.

8. Refrensi

Abraham, A., Pedregosa, F., Eickenberg, M., Gervais, P., Mueller, A., Kossaifi, J., Gramfort, A.,

Thirion, B., & Varoquaux, G. (2014). Machine learning for neuroimaging with scikit-learn.

Frontiers in Neuroinformatics, 8. <https://doi.org/10.3389/fninf.2014.00014>

Nursakinah, B. (t.t.). *MACHINE LEARNING MENGGUNAKAN PYTHON* :